Laboratorium nr 3 — SQL kontra Apache Solr

Porównanie wyszukiwania pełnotekstowego, facetów, skalowania i rozszerzeń (Poziom 5.0)

Autor: Michał | **Poziom realizacji:** 5.0 (pełny zakres: 3.0 + 4.0 + 5.0) **Środowisko:** PostgreSQL 17, Apache Solr 10 (`SOLR_HEAP=2g`), Python 3.12.3

Sprzęt pomiarowy: Ubuntu 24.04.5 LTS (Docker), procesor Intel Core i5-11400H (6 rdzeni / 12 wątków), 32 GB RAM, dysk Samsung NVMe SSD.

1. Zbiór danych i weryfikacja jakości

Wykorzystano zbiór **500 000 artykułów** z angielskiej Wikipedii: Hugging Face: wikimedia/structured-wikipedia (konfiguracja `enwiki_namespace_0`, licencja **CC BY-SA 4.0**). Plik źródłowy `data-set/wikipedia.csv` zajmuje **843 MiB**. Długość tekstu artykułu została ucięta do maks. 5 000 znaków, aby pojedyncze gigantyczne artykuły nie zaburzały pomiarów.

Parametr zbioru	Wartość zmierzona	Wymaganie z instrukcji (Poziom 4.0 / 5.0)	Status
Liczba rekordów / unikalne ID	500 000 / 500 000	Minimum 500 000 rekordów, unikalny klucz	SPEŁNIONE
Długość tekstu naturalnego (`text`)	średnio 1 686 zn. (min. 300, max. 5 000)	Średnio min. 300 znaków tekstu naturalnego	SPEŁNIONE
Pola kategoryczne (do filtrów i facetów)	`topic` (7 grup), `article_type` (6 grup)	Co najmniej 2 pola kategoryczne	SPEŁNIONE
Pole daty / liczbowe (do zakresów i sortowania)	`date_modified` (500 000 poprawnych dat)	Co najmniej 1 pole liczbowe lub data	SPEŁNIONE

> **Przykładowy rekord:** `id = 72535323` | `title = "Khan gizi" spring` | `topic = geography` | `article_type = location` | `date_modified = 2023-11-14T18:22:10Z`
> **Początek pola** `text` **:** *„Khan gizi spring is a spring located next to Khan's daughter Natevan's palace in the Çöl Qala neighborhood of Shusha..."*

2. Schemat bazy PostgreSQL i konfiguracja Apache Solr

2.1. Schemat tabeli i indeksu pełnotekstowego w PostgreSQL (`database/01_schema.sql`)

W PostgreSQL utworzono tabelę `wikipedia_articles`. Wyszukiwanie pełnotekstowe wykorzystuje kolumnę `search_vector` typu `TSVECTOR` (budowaną z połączenia tytułu i treści przy użyciu słownika `english`) oraz indeks odwrócony **GIN**: `CREATE INDEX idx_wikipedia_articles_search ON wikipedia_articles USING GIN (search_vector);`

Kolumna	Typ SQL	Ograniczenia	Rola w zapytaniach i indeksowaniu
`id`	BIGINT	PRIMARY KEY	Unikalny klucz główny artykułu (indeks B-Tree); służy do porównywania wyników z Solrem.
`title`	TEXT	NOT NULL	Tytuł artykułu; wchodzi do wektora `search_vector` oraz testów `LIKE` i autouzupełniania.
`text`	TEXT	NOT NULL	Treść artykułu (300–5 000 znaków); wchodzi do wektora `search_vector` i testu `LIKE '%run%'` .
`topic`	TEXT	NOT NULL	Kategoria tematyczna (7 wartości: *other, sports, geography, history, culture, politics, science*) – używana w `GROUP BY` .
`article_type`	TEXT	NOT NULL	Typ artykułu (6 wartości: *general, person, event, media, organization, location*) – używany w `GROUP BY` .
`date_modified`	TIMESTAMP	NOT NULL	Data ostatniej modyfikacji; grupowana w przedziały 5-letnie (2000–2030).
`search_vector`	TSVECTOR	NOT NULL	Przechowuje znormalizowane rdzenie słów (leksemy) z pozycjami; indeksowana indeksem **GIN** pod operator `@@` .

2.2. Schemat Apache Solr i uzasadnienie flag pól (`solr/schema.json`)

W core `wikipedia` **wyłączono tryb schemaless** (`update.autoCreateFields=false`), aby Solr nie zgadywał typów jako nieoptymalnych pól wielowartościowych. Wszystkie pola zdefiniowano jawnie przez Schema API:

Pole w Solr	Typ pola	indexed	stored	docValues	multiValued	Uzasadnienie konfiguracji flag i łańcucha analizy
`id`	`string`	true	true	true	false	Klucz unikalny (`uniqueKey`); wymaga szybkiego wyszukiwania, zwracania ID oraz sortowania w `cursorMark` .
`title`	`text_general`	true	true	false	false	Analizowany tekst (tokenizacja, małe litery, stoplista). `docValues=false` , bo pola dzielone na tokeny nie używają pamięci kolumnowej.
`text`	`text_general`	true	true	false	false	Treść artykułu; `stored=true` jest wymagane, aby działało podświetlanie fragmentów trafień (Highlighting `hl=true`).
`topic`	`string`	true	true	true	false	Nierozbijany ciąg znaków. `docValues=true` tworzy strukturę kolumnową w RAM-ie, przyspieszając facety kilkadziesiąt razy.
`article_type`	`string`	true	true	true	false	Druga kategoria do facetów (`facet.field`); jak wyżej – `docValues=true` zapewnia błyskawiczną agregację.
`date_modified`	`pdate`	true	true	true	false	Typ daty oparty na drzewie KD (`PointDateField`) z `docValues=true` – zoptymalizowany pod zapytania zakresowe `facet.range` .
`title_prefix`	`title_prefix_type`	true	false	false	false	Kopia pola `title` (`copyField`) pod autouzupełnianie (4.0). `stored=false` oszczędza dysk – potrzebujemy tylko indeksu prefiksów.

3. Pomiar 1: Czas ładowania danych i budowy indeksów (500 000 rekordów)

W PostgreSQL zmierzono osobno szybki import surowego pliku CSV komendą `COPY` oraz czas wyliczenia wektorów `tsvector` i budowy indeksu odwróconego `GIN`. W Solr skrypt `solr/load_data.py` wysyłał paczki dokumentów JSON przez `/update` z jednym `commit` na końcu (obejmując analizę tekstu i generowanie prefiksów Edge-NGram).

System i etap	Liczba rekordów	Czas (ms)	Czas (s)	Co dzieje się pod maską?
PostgreSQL — sam import `COPY`	500 000	11 512 ms	11,512 s	Zwykłe zrzucenie wierszy tekstu na dysk bez analizy słów.
PostgreSQL — budowa `tsvector` **+ indeks** `GIN`	500 000	202 987 ms	202,987 s	Tokenizacja 500k artykułów, stemming i budowa drzewa odwróconego GIN.
PostgreSQL — ŁĄCZNIE (Import + Indeks GIN)	500 000	214 499 ms	214,499 s	Pełny czas do uzyskania gotowej wyszukiwarki w SQL (~3 min 34 s).
Apache Solr — Import JSON + Indeksowanie + Commit	500 000	83 024 ms	83,024 s	Wgranie paczek, analiza tekstu, budowa `docValues` i `EdgeNGram` (~1 min 23 s).



4. Pomiar 2: Porównanie 3 zapytań, LIKE oraz rankingu Top-10 (Poziom 3.0 i 4.0)

4.1. Wyniki 3 zapytań pełnotekstowych i porównanie z `LIKE '%run%'`

W obu systemach wykonano te same 3 zapytania pełnotekstowe: pojedyncze słowo (`run`), dokładną frazę (`"United States"`) oraz wyrażenie logiczne z negacją (`history AND (war OR peace) AND NOT fiction`). Dodatkowo zapytanie nr 1 wykonano w SQL przez `LIKE '%run%'` bez użycia indeksu GIN.

Scenariusz	Dokładne zapytanie SQL / parametry URL Solra	Wyniki SQL	Solr numFound	Czas SQL	Czas Solr (QTime / klient)
1a. Słowo (Full-Text) `run`	**SQL:** `search_vector @@ plainto_tsquery('english', 'run')` **Solr:** `q=run&defType=edismax&qf=title^3+text`	28 800	14 492	53,311 ms*	**0 ms / 2,203 ms**
1b. Słowo (Skan) `LIKE '%run%'`	**SQL:** `title LIKE '%run%' OR text LIKE '%run%'` (Brak wsparcia indeksu GIN – pełny skan tabeli)	48 804	—	757,659 ms*	—
2. Dokładna fraza `"United States"`	**SQL:** `search_vector @@ phraseto_tsquery('english', 'United States')` **Solr:** `q="United+States"&defType=edismax&qf=title^3+text`	38 148	38 188	222,591 ms	**1 ms / ~2,5 ms**
3. Logika z negacją `AND / OR / NOT`	**SQL:** `to_tsquery('english', 'history & (war | peace) & !fiction')` **Solr:** `q=(history+AND+(war+OR+peace)+AND+NOT+fiction)&defType=edismax`	4 240	3 984	13,238 ms	**1 ms / ~2,3 ms**

* Czasy dla zapytania nr 1 (Full-Text i LIKE) to mediany z 5 uruchomień na rozgrzanym cache'u (z sekcji 5); dla zapytań 2 i 3 czasy z drugiego uruchomienia.

Skąd wynikają różnice w liczbie znalezionych dokumentów? (Komentarz do wyników):

1. **Dlaczego** `LIKE '%run%'` **znalazł aż 48 804 artykuły?** Ponieważ dopasowuje dowolny podciąg znaków wewnątrz innych słów (np. *grun**t**, p**run**e, B**run**o*), ale z kolei jest czuły na wielkość liter (pomija *Run* na początku zdania).
2. **Dlaczego SQL GIN znalazł 28 800, a Solr 14 492 dla słowa** `run` **?** PostgreSQL użył słownika `english` ze stemmerem Snowball, który sprowadza odmiany (*running, runs, runner*) do wspólnego rdzenia `run` . W Solr pole `text_general` wykonuje tokenizację i zamianę na małe litery (z zachowaniem stoplisty), ale bez agresywnego ucinania końcówek, więc dopasowuje dokładną formę `run` .
3. **Dlaczego dla frazy** `"United States"` **wyniki są niemal identyczne (38 148 vs 38 188)?** Ponieważ nazwa własna rzadko podlega odmianie; drobna różnica 40 dokumentów wynika wyłącznie z innego podziału tekstu na tokeny przy znakach interpunkcyjnych (np. myślnikach i ukośnikach).

4.2. Porównanie rankingu Top-10 (SQL `ts_rank_cd` kontra Solr `eDisMax + BM25`) oraz podświetlanie (`hl`)

Dla wszystkich 3 zapytań pobrano 10 najwyżej ocenionych artykułów z PostgreSQL (sortowanie po `ts_rank_cd(search_vector, query) DESC`) oraz z Solra (`defType=edismax&qf=title^3 text&sort=score desc`) i porównano ich identyfikatory skryptem `solr/compare_top10.py` :

Zapytanie	Przykładowe Top-3 ID z PostgreSQL (`ts_rank_cd`)	Przykładowe Top-3 ID z Apache Solr (`BM25 + title^3`)	Wspólne ID w Top-10
1. Słowo: `run`	`907408` , `46836177` , `50911059`	`41255739` (score 14.39), `41572680` (13.16), `14048326` (12.12)	**0 / 10 (0%)**
2. Fraza: `"United States"`	`29893666` , `72301419` , `15242353`	`22227097` (score 16.22), `11600348` (14.27), `12844936` (14.27)	**0 / 10 (0%)**
3. Logika: `history...`	`17748565` , `37674393` , `22454351`	`36741752` (score 16.35), `41035165` (14.73), `24365992` (14.10)	**0 / 10 (0%)**

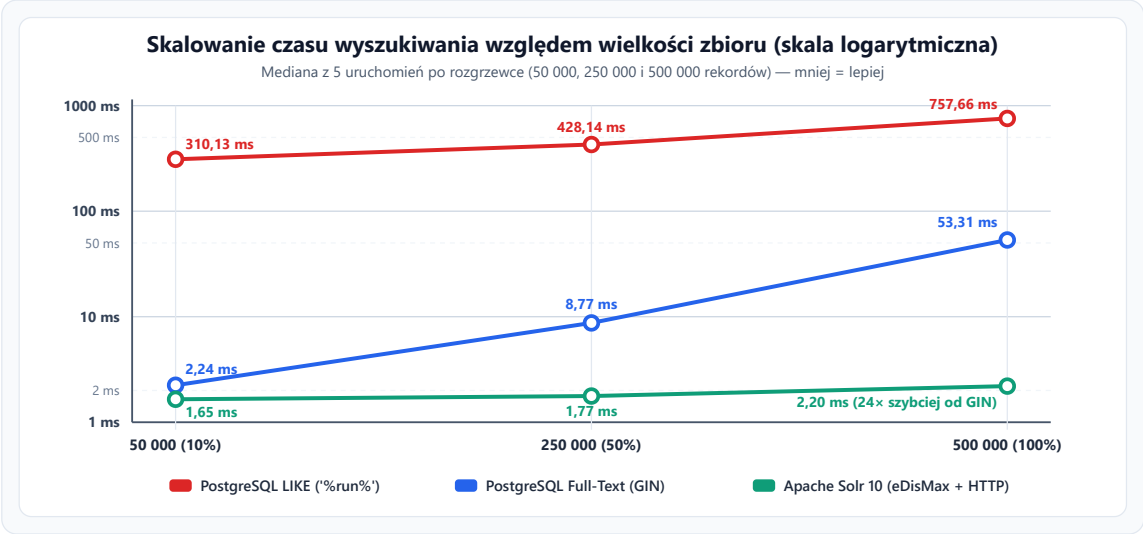
Dlaczego pokrycie Top-10 wynosi 0/10? Oba systemy znalazły tysiące trafnych artykułów, ale **używają zupełnie innych wzorów matematycznych do oceny trafności**:

- **PostgreSQL (`ts_rank_cd`):** ocenia wyłącznie tzw. *Cover Density* (jak gęsto i często szukane słowa leżą obok siebie w tekście), traktując tytuł i treść z taką samą wagą i nie biorąc pod uwagę rzadkości słów w całej bazie.
- **Apache Solr (`eDisMax + BM25`):** nadaje **potrójną wagę trafieniom w tytule (`title^3`)**, promuje krótsze, skoncentrowane dokumenty (normalizacja długości w BM25) oraz uwzględnia rzadkość słów w całym korpusie (IDF). Dodatkowo w zapytaniu nr 4 włączono **podświetlanie trafień** (`hl=true&hl.fl=text&hl.simple.pre=<em>&hl.simple.post=</em>`), co zwraca gotowe wycinki tekstu z wyróżnionym szukanym słowem.

5. Pomiar 3: Skalowanie wyszukiwania — 50 000, 250 000 i 500 000 rekordów (Poziom 5.0)

Aby porównanie było w 100% uczciwe, utworzono deterministyczne podzbiory zawierające **dokładnie te same identyfikatory artykułów** w obu systemach: `50 000` (10% zbioru), `250 000` (50% zbioru) oraz `500 000` (100% zbioru). Każdy test poprzedzono rozgrzewką pamięci podręcznej (*warmup*), a następnie wykonano **5 pomiarów** i wyznaczono **medianę**.

Wielkość zbioru	Metoda wyszukiwania	Dopasowane rekordy	Wyniki 5 kolejnych uruchomień (ms)	Solr QTime (ms)	Mediana czasu (ms)
50 000 rekordów (10% danych)	PostgreSQL `LIKE '%run%'`	6 573	`309,76; 312,94; 332,56; 310,13; 303,37`	—	310,127 ms
	PostgreSQL Full-Text (`GIN`)	4 290	`7,56; 5,06; 2,24; 1,95; 1,85`	—	2,240 ms
	Apache Solr (`eDisMax` , klient HTTP)	2 180	`1,65; 1,69; 1,61; 1,67; 1,64`	`0; 0; 0; 0; 0`	**1,652 ms**
250 000 rekordów (50% danych)	PostgreSQL `LIKE '%run%'`	26 595	`432,14; 417,11; 426,16; 431,08; 428,14`	—	428,139 ms
	PostgreSQL Full-Text (`GIN`)	16 387	`9,56; 8,85; 8,36; 8,77; 8,64`	—	8,765 ms
	Apache Solr (`eDisMax` , klient HTTP)	8 237	`1,82; 1,81; 1,77; 1,72; 1,70`	`0; 0; 0; 0; 0`	**1,767 ms**
500 000 rekordów (100% danych)	PostgreSQL `LIKE '%run%'`	48 804	`781,50; 755,77; 755,68; 757,66; 780,53`	—	757,659 ms
	PostgreSQL Full-Text (`GIN`)	28 800	`53,60; 53,15; 54,26; 53,31; 53,21`	—	53,311 ms
	Apache Solr (`eDisMax` , klient HTTP)	14 492	`1,96; 1,89; 2,53; 2,31; 2,20`	`0; 0; 1; 0; 0`	**2,203 ms**



Wnioski ze skalowania (kiedy który sposób przestaje wystarczać):

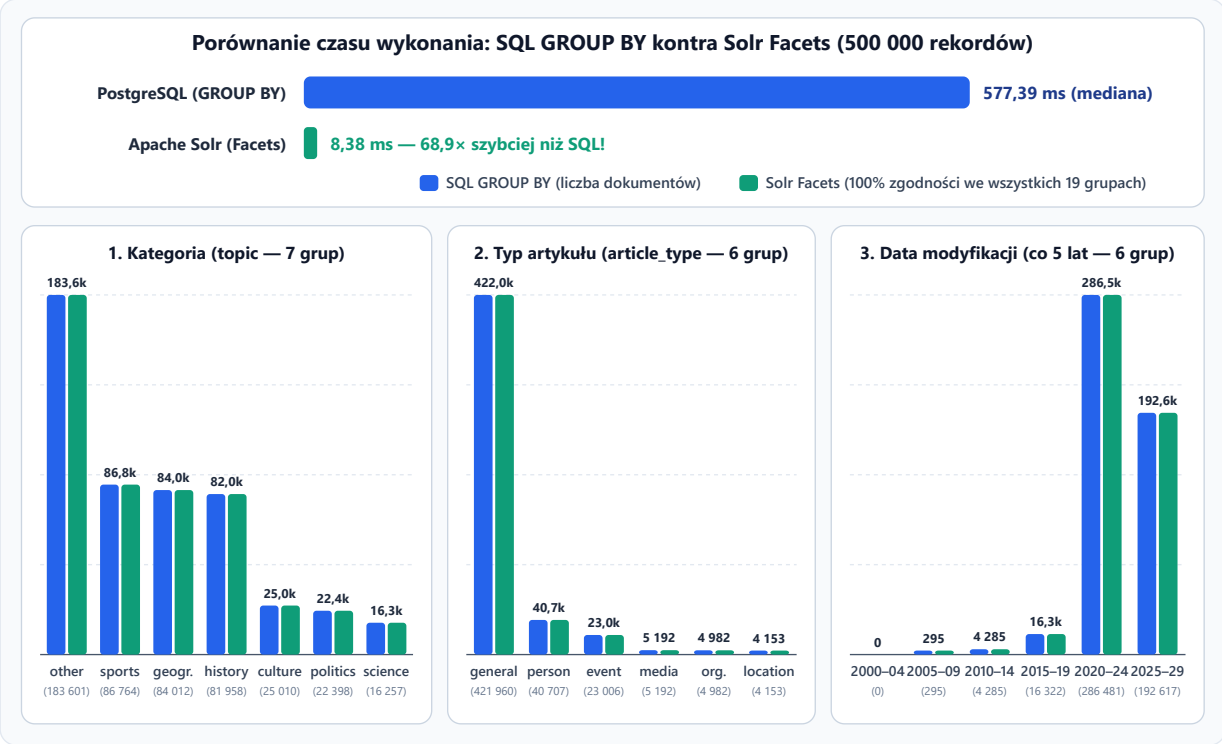
- `SQL LIKE '%...%'` **przestaje wystarczać już przy 50 000 rekordów** (ponad 310 ms na pojedyncze zapytanie, a przy 500 000 aż 758 ms), ponieważ za każdym razem musi przeczytać z pamięci wszystkie wiersze tabeli od początku do końca (*Sequential Scan*).
- `PostgreSQL Full-Text (GIN)` dla 50k–250k rekordów działa bardzo dobrze (2,2–8,8 ms), ale przy 500 000 rekordów czas rośnie **24-krotnie** (do **53,3 ms**), ponieważ lista ID dla popularnego słowa `run` (28 800 trafień) wymaga odczytu i złączenia wielu stron bitmapowych.
- `Apache Solr` wykazuje niemal płaską krzywą skalowania (**1,65 ms → 1,77 ms → 2,20 ms**; wewnątrz serwera `QTime = 0–1 ms`). Nawet z narzutem komunikacji HTTP Solr przy 500k rekordów jest **24 razy szybszy od indeksu GIN** i **344 razy szybszy od LIKE**.

6. Pomiar 4: Agregacje `GROUP BY` w SQL kontra `Facety` w Solr (Poziom 4.0 i 5.0)

W obu systemach wykonano jednoczesną agregację całego zbioru **500 000 artykułów** po dwóch polach kategorycznych (`topic` – 7 grup, `article_type` – 6 grup) oraz jednym zakresie dat (`date_modified` w 6 przedziałach 5-letnich od 2000 do 2030 roku — łącznie **19 grup**).

> **Zapytanie w Solr (`solr/facets.sh`):**
> `GET /solr/wikipedia/select?q=*:*&rows=0&facet=true&facet.field=topic&facet.field=article_type&facet.mincount=0&facet.range=date_modified&f.date_modified.facet.range.start=2000-01-01T00:00:00Z&f.date_modified.facet.range.end=2030-01-01T00:00:00Z&f.date_modified.facet.range.gap=%2B5YEARS`
> **Zapytanie w PostgreSQL (`database/07_facets_group_by.sql`):**
> Trzy zapytania `SELECT ... count(*) GROUP BY ...` połączone przez `UNION ALL` (w tym `generate_series('2000-01-01', '2025-01-01', interval '5 years')` dla koszyków czasowych).

Metoda agregacji (500 000 dokumentów, 19 grup)	Wyniki 5 kolejnych uruchomień po rozgrzewce (ms)	Mediana czasu	Stosunek prędkości
PostgreSQL — `GROUP BY` (3 wymiary razem)	`603,291; 558,830; 577,391; 551,552; 578,078`	577,391 ms	1,0× (baza odniesienia)
Apache Solr — 2× `facet.field` + 1× `facet.range`	`8,488; 8,325; 8,688; 8,377; 8,094`	**8,377 ms**	**68,9× szybciej!**



Dlaczego Solr wyliczył wszystkie 19 grup w 8,4 ms, a PostgreSQL potrzebował aż 577,4 ms?

- **100% zgodności wyników:** W każdej z 19 grup (np. *other: 183 601*, *sports: 86 764*, *general: 421 960*, *2020–2024: 286 481*) liczba dokumentów z SQL i Solra zgadza się co do jednego rekordu (różnica = 0).
- **Różnica architektoniczna:** PostgreSQL przechowuje dane **wierszowo**, więc przy `GROUP BY` musi przeskanować wiersze tabeli i zliczać wartości w tablicach haszujących. Solr dzięki fladze `docValues=true` trzyma wartości tych trzech kolumn w zwartych tablicach kolumnowych w pamięci RAM, więc zliczanie polega na szybkim przejściu po indeksach w pamięci podręcznej.

7. Pomiar 5: Rozszerzenia dodatkowe na ocenę 4.0 i 5.0

7.1. Autouzupełnianie (Poziom 4.0): Solr `Edge-NGram` kontra SQL `LIKE 'foot%'`

W Solr utworzono pole `title_prefix` z analizatorem `KeywordTokenizer → LowerCaseFilter → EdgeNGramFilter (1..50)`. Podczas indeksowania każdy tytuł jest z góry dzielony na wszystkie swoje prefiksy od lewej strony (np. dla *Football* powstają wpisy: `f`, `fo`, `foo`, `foot`, `footb`...). W PostgreSQL wykonano zapytanie `WHERE lower(title) LIKE lower('foot%')`.

Metoda autouzupełniania (prefiks `foot`)	Dokładne zapytanie	Liczba trafień	5 kolejnych pomiarów (ms)	Mediana czasu
PostgreSQL — `LIKE 'foot%'`	`WHERE lower(title) LIKE lower('foot%')`	113	`210,52; 193,67; 192,87; 194,38; 207,80`	194,380 ms
Apache Solr — `Edge-NGram`	`q=title_prefix:foot&rows=0`	113	`2,22; 2,05; 2,49; 2,05; 1,87`	**2,051 ms (94,8× szybciej)**

Wniosek: Obie metody zwróciły identyczne **113 artykułów**. PostgreSQL musiał w trakcie zapytania przeczytać 500 000 tytułów i wykonać na każdym funkcję `lower()` oraz porównanie tekstowe (194,4 ms). Solr wykonał pracę podczas indeksowania, więc w momencie zapytania sięgnął bezpośrednio po gotowy klucz `foot` ze słownika indeksu w **2,05 ms**.

7.2. Głębokie stronicowanie (Poziom 5.0): `start/rows` kontra `cursorMark`

Przeprowadzono dwa eksperymenty w Solr (przy wymaganym sortowaniu po unikalnym kluczu `sort=id asc`): **(1)** pobranie dalekiej strony na pozycji **offset 100 000** (1 000 dokumentów) obiema metodami oraz **(2)** pełne pobranie wszystkich **500 000 dokumentów** za pomocą `cursorMark`.

Eksperyment stronicowania	Metoda i parametry URL	Liczba żądań HTTP	Pobrane dokumenty	Mediana czasu całkowitego	Średni czas 1 żądania
Skok do offsetu 100 000 (5 powtórzeń, weryfikacja ID)	**Klasyczny offset:** `start=100000&rows=1000&sort=id+asc`	1	1 000 (100% zgodne ID)	376,914 ms	376,914 ms / żądanie
Dojście do offsetu 100 000 (5 powtórzeń, weryfikacja ID)	**Kursor (zakładka):** `cursorMark=* → nextCursorMark`	101	1 000 (100% zgodne ID)	1 362,369 ms	**13,489 ms / żądanie**
Pełny eksport całej bazy (`solr/cursor_mark.py`)	**Pełny skan** `cursorMark` **:** `rows=1000&sort=id+asc`	501 stron	**500 000** (cały zbiór)	**8 199,099 ms (8,20 s łącznie)**	**16,302 ms (mediana strony)**

Analiza wyników stronicowania (dlaczego `cursorMark` jest niezbędny mimo 1 362 ms przy dojściu do 100k):

- **Koszt pojedynczego zapytania:** W `start=100000` klient wysłał tylko 1 żądanie HTTP, ale trwało ono aż **376,9 ms**, ponieważ Solr musiał w pamięci RAM posortować 101 000 rekordów i odrzucić pierwsze 100 000. W `cursorMark` kursor nie może przeskoczyć w ciemno do środka zbioru – musiał wykonać 101 kroków po sieci HTTP, ale **każde pojedyncze zapytanie trwało tylko 13,5 ms** (28× szybciej na serwerze!).
- **Problem eksportu całego zbioru (złożoność $O(N^2)$ kontra $O(N)$):** Próba pobrania wszystkich 500 stron przez `start/rows` zmusza Solra do sortowania łącznie ponad 125 milionów pozycji w pamięci i kończy się błędem `OutOfMemoryError` oraz duplikatami przy modyfikacji danych. Za pomocą `cursorMark` pobraliśmy **wszystkie 500 000 dokumentów w 8,2 sekundy** przy stałym czasie ~16,3 ms na stronę i minimalnym zużyciu RAM-u.

7.3. Aktualizacja danych SQL → Solr (Poziom 5.0): Wzorzec Transactional Outbox

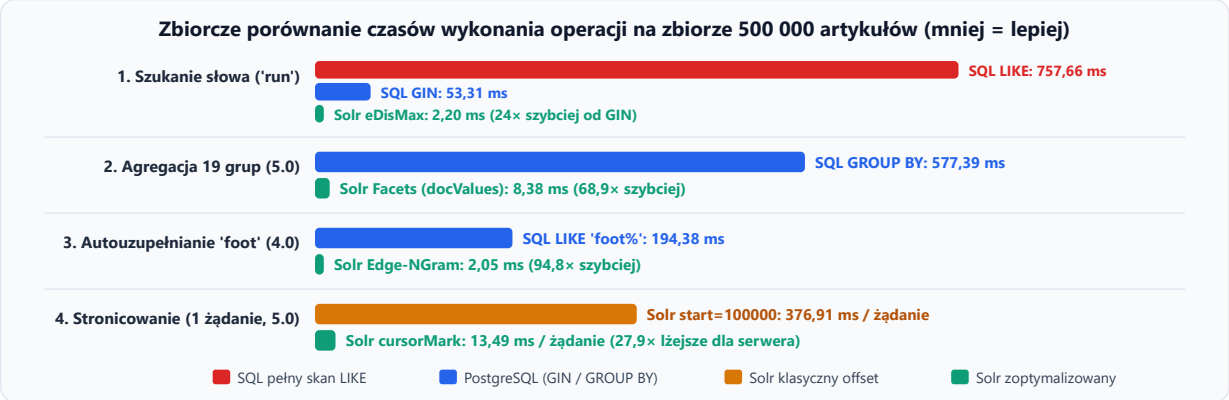
Zrealizowano automatyczne przenoszenie zmian z PostgreSQL do Solra za pomocą produkcyjnego wzorca **Transactional Outbox**:

1. **Trigger w PostgreSQL (** `database/09_sync_outbox.sql` **):** Wyzwalacz `AFTER INSERT OR UPDATE OR DELETE` na tabeli `wikipedia_articles` w tej samej transakcji ACID dopisuje zdarzenie (`upsert` z pełnym dokumentem JSON lub `delete` z ID) do tabeli kolejki `solr_sync_outbox` (gdzie `synced_at IS NULL`).
2. **Skrypt synchronizujący (** `solr/sync_outbox.py` **):** Pobiera oczekujące zdarzenia z tabeli outbox, wysyła je do Solra przez `/update`, wykonuje `<commit/>` i oznacza wiersze w SQL jako zsynchronizowane (`synced_at = clock_timestamp()`). Może działać jednorazowo lub w pętli ciągłej (`--watch`).

Etap testu integracyjnego (`solr/test_sync.py`)	Wykonana operacja w PostgreSQL	Działanie skryptu `sync_outbox.py`	Wynik weryfikacji w Apache Solr
1. Dodanie rekordów (INSERT)	Wstawienie 4 nowych artykułów testowych	Wysłanie 4 zdarzeń `upsert` + `commit`	**ZGODNE (4/4)** Wszystkie pola w Solr identyczne jak w SQL.
2. Edycja rekordów (UPDATE)	Zmiana tytułu, treści i kategorii 2 artykułów	Wysłanie 2 zdarzeń `upsert` + `commit`	**ZGODNE (4/4)** Nowe wartości widoczne w Solr, pozostałe 2 bez zmian.
3. Usunięcie rekordów (DELETE)	Skasowanie 2 artykułów z tabeli SQL	Wysłanie 2 zdarzeń `delete` + `commit`	**ZGODNE (2/2)** Usunięte ID zwracają 0 wyników, 2 pozostałe nienaruszone.
4. Czyszczenie po teście	Usunięcie pozostałych 2 rekordów testowych	Synchronizacja usunięcia + czyszczenie outboxa	**CZYSTO** Przywrócono dokładny stan 500 000 artykułów.

8. Podsumowanie pomiarów i wnioski końcowe (PostgreSQL kontra Apache Solr)

Poniższy wykres zestawia mediany czasów wszystkich zbadanych operacji na pełnym zbiorze **500 000 artykułów**, pokazując skalę przyspieszenia uzyskaną dzięki strukturom indeksowym Apache Solr:



Kryterium / Zastosowanie	Kiedy wystarczy PostgreSQL z indeksem GIN?	Kiedy należy wdrożyć Apache Solr?
Proste wyszukiwanie pełnotekstowe i spójność transakcyjna	Gdy zbiór ma do **100k–250k rekordów** (czas zapytań GIN wynosi wtedy **2,2–8,8 ms**), a priorytetem jest prostota architektury (jedna baza danych) i natychmiastowa spójność ACID.	Gdy zbiór przekracza **500 000 rekordów** lub ruch jest duży – przy 500k czas GIN rośnie do **53,3 ms**, podczas gdy Solr utrzymuje stałe **~2,2 ms** (24× szybciej).
Filtry dynamiczne, liczniki kategorii i zakresów (Faceting)	Gdy aplikacja nie wyświetla liczników kategorii przy każdym zapytaniu – `GROUP BY` na 500k wierszy zajmuje aż **577,4 ms**, co spowalnia interfejs.	W sklepach e-commerce i portalach z filtrami bocznymi – dzięki kolumnowym `docValues` Solr wylicza 19 grup kategorycznych i zakresowych w **8,4 ms** (**69× szybciej**).
Autouzupełnianie na żywo (Autocomplete podczas pisania)	Tylko przy małych tabelach – skan `lower(title) LIKE 'foot%'` na 500k wierszy trwa **194,4 ms**, co powoduje wyczuwalne opóźnienie podpowiedzi.	Gdy wymagane są natychmiastowe podpowiedzi po każdej literze – filtr `Edge-NGram` w Solr zwraca wyniki w **2,05 ms** (**95× szybciej**).
Rekomendowana architektura produkcyjna	Najlepszym rozwiązaniem dla dużych systemów jest **architektura hybrydowa** zrealizowana w tym projekcie: **PostgreSQL** pełni rolę głównego, bezpiecznego magazynu transakcyjnego (*Single Source of Truth*), natomiast **Apache Solr** obsługuje wyszukiwanie, autouzupełnianie, podświetlanie trafień i facety, odbierając aktualizacje asynchronicznie przez kolejkę **Transactional Outbox**.	